Lab 06 — Praktijklab: gegevens hun container laten overleven

Doel: één keer gegevens verliezen om het te begrijpen, en ze daarna laten overleven met een named volume, een bind mount en een tmpfs; met eigen ogen zien van wie de bestanden zijn in rootless; en een PostgreSQL-database op twee manieren back-uppen en terugzetten.

Vereisten — Labs 01 tot 05 afgewerkt. De images `alpine` en `nginx:alpine` staan klaar; `postgres:16-alpine` wordt in stap 2 opgehaald (297 MB).

Meegeleverde bestanden (`files/`) — geen: alles wordt aan de terminal aangemaakt.

De uitvoer hieronder komt van een Ubuntu/WSL met het standaard `subuid`-bereik (`100000:65536`). Container-ID's, datums en hashes verschillen bij jou.

Stap 1 — Een gegeven verliezen

```
mkdir -p ~/labo-docker/06 && cd ~/labo-docker/06
podman run --name scribe alpine sh -c 'echo "notitie van maandag" > /data.txt; ls -l /data.txt'
podman diff scribe
```

Observeer `-rw-r--r-- 1 root root 20 … /data.txt`, en daarna `A /data.txt` en `C /etc`: de writable layer van de container `scribe` bevat een toegevoegd bestand en een gewijzigde map.

```
podman run --rm alpine ls /data.txt
podman start -a scribe
podman rm scribe
```

Observeer `ls: /data.txt: No such file or directory` voor de nieuwe container: de image is niet veranderd. Daarna toont de herstarte `scribe` zijn bestand wel: `stop` / `start` bewaren de laag. Dan `rm`: het bestand bestaat nergens meer.

Uitleg. Elke container heeft zijn eigen writable layer, onzichtbaar voor de andere, en vernietigd samen met hem. Een gegeven dat moet blijven, kan daar niet wonen.

Stap 2 — Een named volume voor PostgreSQL

```
podman volume create pgdata
podman volume inspect pgdata
```

Observeer `"Mountpoint": "/home/<jij>/.local/share/containers/storage/volumes/pgdata/_data"`, `"NeedsCopyUp": true`, `"MountCount": 0`: een lege map, beheerd door Podman, nog nergens gemount.

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman logs db 2>&1 | tail -1
podman exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT INTO
clients(nom) VALUES ('Durand'),('Peeters');"
podman exec db psql -U postgres -c "SELECT * FROM clients;"
podman inspect db --format '{{json .Mounts}}'
```

Observeer `database system is ready to accept connections`, `CREATE TABLE` / `INSERT 0 2`, de twee rijen, en dan de mount: `"Type":"volume","Name":"pgdata",…` `"Destination":"/var/lib/postgresql/data","RW":true`.

```
podman rm -f -t 0 db
podman volume ls
podman run -d --name db2 -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db2 psql -U postgres -c "SELECT * FROM clients;"
podman rm -f -t 0 db2
```

Observeer dat `pgdata` na het verwijderen van de container nog in de lijst staat, en dat de tweede container `Durand` en `Peeters` terugvindt.

Uitleg. De container is weg, het volume is gebleven. De tweede container vond een `PGDATA` die al geïnitialiseerd was en heeft niets opnieuw aangemaakt — `POSTGRES_PASSWORD` heeft alleen effect bij de allereerste start.

Stap 3 — Het anonieme volume waar je niet om vroeg

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'
podman run -d --name anon -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman volume ls
podman inspect anon --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination}}{{end}}'
podman rm -f -t 0 anon
podman volume ls
```

Observeer `map[/var/lib/postgresql/data:{}]`: de image declareert een `VOLUME`. Zonder `-v` heeft Podman een volume aangemaakt met een naam van 64 hexadecimale tekens, gemount op dezelfde plaats. Na `rm` is het er **nog altijd**.

```
podman volume ls --filter dangling=true
```

Observeer dat de filter het anonieme volume toont… **én** `pgdata`. "Dangling" betekent "door geen enkele bestaande container gemount" — en sinds het verwijderen van `db2` geldt dat ook voor `pgdata`, dat nochtans je klanten bevat. Die lijst aan `volume rm` doorgeven (of `podman volume prune` draaien) zou ze allebei vernietigen.

```
podman volume rm $(podman volume ls --filter dangling=true -q | grep -v '^pgdata$')
podman run -d --name anon2 -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman rm -f -t 0 -v anon2
podman volume ls
```

Observeer dat alleen `pgdata` overblijft: het eerste anonieme volume is bij naam verwijderd, het tweede samen met zijn container dankzij `rm -v`.

Uitleg. `podman rm` verwijdert geen volumes; `podman rm -v` verwijdert de **anonieme** volumes van de container (nooit de named). De filter `dangling=true` maakt geen onderscheid tussen afval en een kostbaar volume dat tussen twee containers wacht: lees de lijst, en verwijder dan bij naam.

Stap 4 — Van wie zijn de bestanden?

```
ls -ln ~/.local/share/containers/storage/volumes/
ls ~/.local/share/containers/storage/volumes/pgdata/_data
```

Observeer `drwx------ 3 1000 1000 … pgdata`: de map van het volume is van jou. Maar de inhoud: `ls: cannot open directory … Permission denied`.

```
podman unshare cat /proc/self/uid_map
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -4
```

Observeer de tabel `0 1000 1` gevolgd door `1 100000 65536`, en de bestanden `-rw------- 1 70 70 … PG_VERSION`: in de namespace zijn ze van UID 70, `postgres`.

Uitleg. PostgreSQL schrijft als UID 70. Buiten de namespace wordt dat UID vertaald naar `100000 + 70 - 1 = 100069`: een gebruiker die jij niet bent. `podman unshare` voert een commando *in* de namespace uit, met dezelfde kijk als de container. (In de sandbox waarin deze labs geschreven zijn, begint het subuid-bereik bij 1001; het principe is identiek.)

Stap 5 — Een bind mount voor nginx

```
mkdir -p html && echo '<h1>Versie 1</h1>' > html/index.html
podman run -d --name web -p 18080:80 -v ./html:/usr/share/nginx/html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080
echo '<h1>Versie 2</h1>' > html/index.html
curl -s localhost:18080
```

Observeer `<h1>Versie 1</h1>` en daarna `<h1>Versie 2</h1>`: het bestand dat je op de host wijzigt, wordt meteen geserveerd, zonder herstart of rebuild.

```
podman exec web sh -c 'echo "<h1>piraat</h1>" > /usr/share/nginx/html/index.html'
podman inspect web --format '{{json .Mounts}}'
```

Observeer `sh: can't create …/index.html: Read-only file system`, en in de inspect `"Type":"bind","Source":"/home/<jij>/labo-docker/06/html",… "RW":false`.

```
podman run -d --name web-rw -p 18081:80 -v ./html:/usr/share/nginx/html nginx:alpine
podman exec web-rw sh -c 'echo "<h1>Versie 3 (geschreven vanuit de container)</h1>" >
/usr/share/nginx/html/index.html'
cat html/index.html ; ls -ln html/
podman cp web:/etc/nginx/nginx.conf ./nginx.conf && head -3 nginx.conf
podman rm -f -t 0 web web-rw
```

Observeer `Versie 3` in **jouw** bestand, dat nog altijd van `1000 1000` is: geschreven door de root van de container, dus door jou. En `podman cp` heeft de nginx-configuratie eruit gehaald zonder mount.

Uitleg. Een bind mount is dezelfde map, gezien van twee kanten. `:ro` beschermt je bestanden; zonder die optie schrijft de container bij jou, met het UID dat bij zijn gebruiker hoort.

Stap 6 — De UID-valkuil

```
mkdir -p uitvoer
podman run --rm -v ./uitvoer:/out alpine sh -c 'id -u; echo hallo > /out/root.txt'
ls -ln uitvoer
podman run --rm --user 1000 -v ./uitvoer:/out alpine sh -c 'id -u; echo hallo > /out/user.txt'
```

Observeer `0` en een bestand `root.txt` dat van `1000 1000` is. Daarna `1000` en `sh: can't create /out/user.txt: Permission denied`.

```
podman run --rm --user 1000 -v ./uitvoer:/out:U alpine sh -c 'id -u; echo hallo >
/out/user.txt; ls -ln /out'
ls -ln uitvoer
```

Observeer in de container `-rw-r--r-- 1 1000 0 … user.txt`, en op de host `-rw-r--r-- 1 100999 1000 … root.txt` en `user.txt`: `:U` heeft de eigenaar **van de hele map** gewijzigd naar het UID van de container, en dat ben jij niet meer.

```
cat uitvoer/user.txt
podman unshare chown -R 0:0 uitvoer && ls -ln uitvoer
```

Observeer dat lezen lukt (rechten `r` voor iedereen), en daarna `1000 1000`: de map is weer van jou.

Uitleg. De root van de container, dat ben jij; UID 1000 van de container is 100999 op de host. `:U` doet een echte, blijvende `chown`; `podman unshare chown 0:0` draait dat terug, want root in de namespace = 1000 erbuiten.

Stap 7 — tmpfs en de `--mount`-syntaxis

```
podman run --rm --tmpfs /scratch:size=64m alpine sh -c 'df -h /scratch; echo tijdelijk >
/scratch/f.txt; ls /scratch'
podman run --rm --mount type=tmpfs,destination=/scratch,tmpfs-size=64m alpine df -h /scratch
```

Observeer `tmpfs 64.0M 0 64.0M 0% /scratch` en `f.txt` : een schijf in het geheugen van 64 MB, die bij de volgende regel al niet meer bestaat.

```
podman run --rm --mount type=volume,source=pgdata,destination=/data alpine ls /data | head -3
podman run --rm --mount type=bind,source=./html,destination=/html,ro alpine sh -c 'cat
/html/index.html; touch /html/x'
podman run --rm -v ./html:/html:ro,Z alpine cat /html/index.html
```

Observeer `PG_VERSION` , `base` , `global` (het volume gezien door een andere container), `Read-only file system` bij de `touch` , en `:Z` dat aanvaard wordt zonder iets te veranderen.

Uitleg. `--mount` zegt expliciet wat je mount; `-v` leidt het af uit de vorm van de bron. `:Z` doet niets zonder SELinux, maar maakt je commando's klaar voor RHEL-servers.

Stap 8 — Back-uppen, breken, terugzetten

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db pg_dump -U postgres > dump.sql
ls -lh dump.sql ; grep -A3 "COPY public.clients" dump.sql
podman volume rm pgdata
```

Observeer een `dump.sql` van ongeveer 1,8 kB met `COPY public.clients (id, nom) FROM stdin;` gevolgd door de twee rijen: een logische back-up, gemaakt terwijl de database draait. Daarna `Error: volume pgdata is being used by the following container(s): … volume is being used` .

```
podman stop -t 5 db
podman volume export pgdata -o pgdata.tar
ls -lh pgdata.tar ; tar tf pgdata.tar | head -3
podman volume create pgdata-kopie
podman volume import pgdata-kopie pgdata.tar
podman run -d --name db-kopie -e POSTGRES_PASSWORD=secret -v pgdata-
kopie:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-kopie psql -U postgres -c "SELECT * FROM clients;"
```

Observeer een archief van 39 MB (`PG_VERSION` , `base/` , …) en, in het geïmporteerde volume, de twee klanten: een fysieke back-up, met gestopte container.

```
podman volume create pgdata-leeg
podman run -d --name db-leeg -e POSTGRES_PASSWORD=secret -v pgdata-
leeg:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-leeg psql -U postgres -c "SELECT * FROM clients;"
podman exec -i db-leeg psql -U postgres -q < dump.sql
podman exec db-leeg psql -U postgres -c "SELECT * FROM clients;"
```

Observeer `ERROR: relation "clients" does not exist` op de lege database; het afspelen van de dump toont alleen twee kleine resultaten (`set_config` , `setval`); en dan zijn de twee klanten terug.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
ls -lh pgdata.tgz
podman system df -v | sed -n '/Local Volumes/,$p'
```

Observeer `pgdata.tgz` (4,5 MB gecomprimeerd), in jouw map geschreven door een wegwerpcontainer, en de drie volumes van elk ongeveer 40 MB.

Uitleg. `pg_dump` leest een consistente momentopname terwijl de database draait; een bestandskopie vereist een stop. `volume export` / `import` zijn eigen aan Podman; de Alpine-container met twee mounts is de methode die overal werkt — en de enige die de bestanden van UID 100069 leest zonder `unshare`.

Opruimen

```
podman rm -f -t 0 db db-kopie db-leeg
podman volume rm pgdata pgdata-kopie pgdata-leeg
podman volume ls
podman rmi docker.io/library/postgres:16-alpine
rm -rf ~/labo-docker/06/uitvoer ~/labo-docker/06/pgdata.tar ~/labo-docker/06/pgdata.tgz
```

Observeer een lege `podman volume ls`. Hou `html/` en `dump.sql`: de volgende labs hebben ze niet nodig, maar ze nemen amper 2 kB in. `postgres:16-alpine` komt in lab 07 terug — ze hier verwijderen bespaart 297 MB tot dan, en lab 07 ruimt ze opnieuw op.

Wat je nu moet kunnen beweren

- `stop` / `start` bewaren de writable layer; `rm` vernietigt ze: `No such file or directory`.
- Een named volume overleeft zijn container: `db2` vindt de klanten van `db` terug.
- `VOLUME` in de image = anoniem volume van 64 tekens als je `-v` vergeet; `rm -v` neemt het mee, en `dangling=true` toont ook je named volumes die even niet gemount zijn.
- De bestanden van PostgreSQL zijn van `70` in de namespace en van `100069` erbuiten: `Permission denied` zonder `podman unshare`.
- Een bind mount met `:ro` wordt live geserveerd en weigert schrijven: `Read-only file system`.
- `--user 1000` kan niet in jouw map schrijven; `:U` geeft hem de map, `podman unshare chown` neemt ze terug.
- `pg_dump` terwijl de database draait, `volume export` of `tar` met gestopte container; `volume rm` weigert zolang een container het volume mount.